

Projet : Application Web collaborative de gestion de projets (type plans de communication)

Backend Node.js / Express

Dépendances

- bcrypt : hachage mots de passe
- cookie-parser et jsonwebtoken : gestion des cookies http only cryptés
- cors : gestion des autorisations "Cross-origin resource sharing"
- dotenv : gestion des données d'environnement (lien PostgreSQL, ...)
- multer : gestion des uploads d'images
- pg : gestion de la connexion avec la base de données
- nodemon : seulement côté "dev" pour relancer le serveur automatiquement

Fonctionnement général des routes backend

L'application utilise une architecture modulaire avec séparation des responsabilités :

Structure des routes :

- Toutes les routes sont préfixées par `/api/`
- Chaque module métier possède son propre fichier de routes
- Les routes utilisent le pattern factory avec injection de la connexion PostgreSQL
- Middleware d'authentification pour sécuriser les ressources protégées

Routes disponibles :

- `/api/auth` : Authentification et gestion des utilisateurs
- `/api/header` : Gestion des en-têtes
- `/api/goals` : Gestion des objectifs
- `/api/targets` : Gestion des cibles
- `/api/actions` : Gestion des actions
- `/api/tasks` : Gestion des tâches
- `/api/admin` : Routes d'administration

Sécurisation des ressources statiques :

- Images des goals : `/uploads/goals_images` avec middleware `authenticateGoalImage`
- Images des targets : `/uploads/targets_images` avec middleware `authenticateTargetImage`

- Vérification des droits d'accès avant servir les fichiers statiques

Routes d'authentification “auth” et gestion utilisateur

POST /api/auth/register - Inscription utilisateur

- Validation complète des données (email, mot de passe, nom, prénom, date de naissance)
- Règles de validation du mot de passe : minimum 8 caractères avec majuscule, minuscule, chiffre et caractère spécial
- Validation de l'âge : utilisateur doit avoir plus de 12 ans
- Hachage sécurisé du mot de passe avec bcrypt (10 rounds de salt)
- Vérification de l'unicité de l'email

POST /api/auth/login - Connexion utilisateur

- Authentification par email/mot de passe
- Génération d'un JWT signé avec expiration 7 jours
- Stockage du token dans un cookie HTTP Only sécurisé
- Configuration cookie adaptée à l'environnement (secure en production)
- Retour du statut admin de l'utilisateur

POST /api/auth/logout - Déconnexion utilisateur

- Suppression du cookie d'authentification
- Configuration de suppression sécurisée du cookie

GET /api/auth/me - Informations utilisateur connecté

- Middleware d'authentification requis
- Récupération des données fraîches depuis la base de données
- Retour des informations : id, email, statut admin

POST /api/auth/change-password - Changement de mot de passe

- Middleware d'authentification requis
- Validation du mot de passe actuel
- Validation de la complexité du nouveau mot de passe
- Vérification que le nouveau mot de passe est différent de l'ancien
- Mise à jour sécurisée avec hachage bcrypt

Sécurité implémentée :

- Cookies HTTP Only avec sameSite strict
- Hachage bcrypt avec salt pour tous les mots de passe
- Validation stricte des données d'entrée
- Messages d'erreur génériques pour éviter l'énumération d'utilisateurs
- JWT avec expiration et signature sécurisée
- Protection CORS configurée via variables d'environnement

Routes d'administration système

Protection : Double middleware (`authenticateToken` + `requireAdmin`) - accès réservé aux administrateurs uniquement

Endpoints disponibles :

GET `/api/admin/campaigns` - Liste complète des campagnes

- Récupère toutes les campagnes de la plateforme (id, nom, dates de création/modification)
- Tri par date de création décroissante
- Retour avec compteur total des campagnes
- Usage : Vue d'ensemble administrative de toutes les campagnes utilisateurs

GET `/api/admin/stats` - Statistiques globales de la plateforme

- **Métriques calculées :**
 - Nombre total de campagnes
 - Nombre total d'utilisateurs
 - Nombre total d'administrateurs
- Requêtes optimisées avec `COUNT(*)` pour les performances
- Usage : Dashboard administrateur avec KPI de la plateforme

PostgreSQL :

- utilisation de la table "users", si la ligne "admin" est à "true"

Sécurité :

- Accès exclusivement réservé aux comptes avec `admin = true`
- Validation en base de données à chaque requête

Middlewares de vérification d'authentification, d'administration et d'accès aux images

L'application utilise plusieurs middlewares pour sécuriser l'accès aux ressources.

auth.js - Middleware d'authentification de base

Fonction : `authenticateToken`

- Rôle : Vérification de l'authentification utilisateur pour les routes protégées
- Fonctionnement :
 - Récupère le token JWT depuis le cookie `authToken`
 - Vérifie la validité et la signature du token avec `JWT_SECRET`
 - Injecte les informations utilisateur dans `req.user` pour les routes suivantes
 - Retourne 401 (Non authentifié) si pas de token

- Retourne 403 (Accès interdit) si token invalide/expiré
- Usage : Appliqué sur toutes les routes nécessitant une authentification

admin.js - Middleware d'autorisation administrateur

Fonction : **requireAdmin**

- Rôle : Vérification des droits administrateur avec double validation
- Fonctionnement :
 - Première vérification : validation du token JWT
 - Seconde vérification : requête en base de données pour confirmer le statut admin
 - Vérifie l'existence de l'utilisateur en base
 - Contrôle le champ **admin** de la table users
 - Injecte les informations complètes dans **req.user**
- Sécurité : Double validation (token + base de données) pour éviter l'élévation de privilèges
- Usage : Appliqué sur les routes d'administration **/api/admin**

authenticateGoalImage.js - Middleware de sécurisation des images de goals

Fonction : **authenticateGoalImage**

- Rôle : Contrôle d'accès granulaire aux images uploadées pour les objectifs
- Fonctionnement :
 - Intercepte les requêtes vers **/uploads/goals_images/**
 - Extrait le nom de fichier depuis l'URL de la requête
 - Vérifie l'association image-goal via le champ **goals_imageurl**
 - Logique d'autorisation (cascade) :
 1. Propriétaire direct : **goal.user_id === user.id**
 2. Propriétaire de campagne : via table **campaign**
 3. Accès partagé : via table **share_campaigns**
 - Autorise l'accès seulement si une des conditions est remplie
- Usage : Appliqué automatiquement sur le serveur de fichiers statiques

authenticateTargetImage.js - Middleware de sécurisation des images de targets

Fonction : **authenticateTargetImage**

- Rôle : Contrôle d'accès granulaire aux images uploadées pour les cibles
- Fonctionnement : Identique au middleware goal mais pour les targets
 - Intercepte les requêtes vers **/uploads/targets_images/**
 - Vérifie l'association via le champ **targets_imageurl**
 - Même logique d'autorisation en cascade :
 1. Propriétaire direct du target
 2. Propriétaire de la campagne
 3. Utilisateur avec accès partagé
- Usage : Appliqué automatiquement sur le serveur de fichiers statiques

Architecture de sécurité des images

Principe : Aucune image n'est accessible directement, toutes passent par un middleware d'autorisation

Routes centralisées de l'interface utilisateur

Rôle : Hub principal regroupant toutes les fonctionnalités accessibles depuis l'en-tête/navigation de l'application

Architecture : Organisation modulaire avec délégation aux contrôleurs spécialisés

Fonctionnalités regroupées :

Gestion utilisateur :

- **GET /user** : Récupération profil (prénom, nom)
- **PUT /user/profile** : Mise à jour profil utilisateur
- **DELETE /account** : Suppression complète du compte (avec transaction)

Gestion des campagnes :

- **POST /campaign** : Création nouvelle campagne
- **GET /campaigns** : Liste des campagnes (propriétaire + partagées avec rôles)
- **DELETE /campaign/:id** : Suppression campagne avec données cascade
- **DELETE /shared-campaign/:id** : Quitter une campagne partagée

Gestion collaborative :

- **GET /campaign/:id/users** : Liste utilisateurs d'une campagne (propriétaire uniquement)
- **DELETE /campaign/:id/user/:userId** : Expulser un utilisateur (propriétaire uniquement)
- **POST /invite** : Envoyer invitation collaboration (reader/editor)

Gestion des invitations :

- **GET /invitations** : Invitations en attente pour l'utilisateur connecté
- **PUT /invitation/:id/accept** : Accepter invitation
- **DELETE /invitation/:id/reject** : Refuser invitation

Sécurité : Toutes les routes protégées par **authenticateToken** + validation des permissions via services dédiés.

Usage : Interface de gestion globale accessible depuis le header/navbar pour toutes les actions utilisateur principales.

Gestion des objectifs (fichiers “goals”)

Architecture : Pattern Service/Controller avec séparation claire des responsabilités

- **Routes** → **Controller** → **Service** → **Base de données**

Gestion des permissions :

- Système d'accès granulaire par campagne : propriétaire, partagé en lecture/écriture, partagé en lecture seule
- Vérification automatique des droits avant toute opération (CRUD)
- Endpoint dédié : `GET /permissions/:campaignId` pour connaître les droits utilisateur

CRUD complet avec upload d'images :

- **Création** : `POST /upload-image` avec multer pour gestion des images
- **Lecture** : Récupération par ID campagne ou nom campagne
- **Modification** : `PUT /update/:goalId` avec possibilité de changer l'image
- **Suppression** : `DELETE /delete/:goalId` avec vérification des droits

Gestion des images :

- Storage automatique dans `/uploads/goals_images/`
- Nommage unique avec timestamp + random
- Création automatique des dossiers si inexistants
- Sécurisation via middleware d'authentification des images

Système de permissions collaboratives :

- **Propriétaire** : accès total (lecture/écriture/suppression)
- **Partagé écriture** : via table `share_campaigns` avec `read = false`
- **Partagé lecture** : via table `share_campaigns` avec `read = true`
- Gestion des erreurs spécifiques : `READ_ONLY_ACCESS`, `UNAUTHORIZED`, `CAMPAIGN_NOT_FOUND`

Sécurité : Toutes les routes protégées par `authenticateToken` + validation des permissions sur chaque action.

Gestion des cibles (fichiers “targets”)

Même fonctionnement que les objectifs, mais liés cette fois à la table “targets” de PostgreSQL.

Gestion des actions (fichiers “actions”)

Architecture : Pattern Service/Controller avec service centralisé d'accès aux campagnes

- **Routes** → **Controller** → **ActionsService** → **CampaignAccessService** → **Base de données**

Gestion des événements avec permissions :

- **GET /with-access** : Récupération événements + permissions utilisateur (endpoint principal)
- **GET /** : Récupération événements seuls (maintenu pour compatibilité)
- Logique contextuelle : événements de campagne VS événements personnels

CRUD complet des événements :

- **Création** : **POST /** avec validation permissions d'écriture
- **Modification** : **PUT /:id** avec contrôle propriétaire + permissions campagne
- **Suppression** : **DELETE /:id** avec même logique de permissions
- **Format** : id, title, date, startTime, campaignId, userId

Système de permissions avancé :

- **Événements personnels** : accès total pour le créateur (campaignId = null)
- **Événements de campagne** : droits selon rôle dans la campagne
- **Logique collaborative** : propriétaire d'événement + permissions de campagne
- **Double validation** : créateur de l'événement ET droits sur la campagne

Gestion contextuelle :

- Paramètre **currentCampaignId** pour filtrer par contexte
- Récupération de TOUS les événements d'une campagne (peu importe le créateur)
- Événements personnels visibles uniquement par leur créateur

Sécurité :

- Utilisation du **CampaignAccessService** centralisé pour toutes les vérifications
- Validation systématique : **validateCampaignAccess()**, **canWriteToCampaign()**
- Gestion fine des erreurs : **EVENT_NOT_FOUND**, **UNAUTHORIZED**, **CAMPAIGN_NOT_FOUND**cdsc

Gestion des tâches (fichiers "tasks")

Architecture : Pattern Service/Controller avec service centralisé d'accès aux campagnes

- **Routes** → **Controller** → **TasksService** → **CampaignAccessService** → **Base de données**

Fonctionnalités principales :

CRUD complet des tâches :

- **Récupération** : **GET /** avec paramètre **campaignId** obligatoire
- **Création** : **POST /** avec validation permissions d'écriture
- **Modification complète** : **PUT /:id** avec mise à jour de tous les champs
- **Modification statut** : **PATCH /:id** pour changement rapide de statut
- **Suppression** : **DELETE /:id** avec vérification des droits

Structure des tâches :

- **Champs** : id (UUID), title, description, status, assignee, priority, user_id, campaign_id
- **Génération UUID** automatique via `gen_random_uuid()`
- **Traçabilité** : user_id du créateur conservé

Système de permissions strict :

- **Paramètre obligatoire** : campaignId requis pour toutes les opérations
- **Validation systématique** : `validateCampaignAccess()` avec `requireWrite=true`
- **Retour des permissions** : `isReadOnly`, `isOwner` avec les tâches
- **Gestion granulaire** : vérification d'accès pour chaque action CRUD

Gestion d'erreurs spécialisée :

- `CAMPAIGN_NOT_FOUND` → 403 : Accès refusé à la campagne
- `READ_ONLY_ACCESS` → 403 : Permissions insuffisantes
- `Task not found` → 404 : Tâche inexistante
- Messages contextualisés selon l'action (create/update/delete)

Sécurité collaborative :

- Utilisation exclusive du `CampaignAccessService` centralisé
- Aucune tâche accessible sans campagne valide
- Respect des rôles : propriétaire/éditeur/lecteur
- Récupération du campaign_id depuis la tâche pour validation

Test CRUD et d'intégration

Structure des tests

Framework : Jest avec SuperTest pour les tests d'intégration HTTP **Architecture** : Tests unitaires et d'intégration avec mocks PostgreSQL

Configuration générale

`jest.config.js` - Configuration Jest :

- Environnement Node.js
- Timeout 10 secondes pour tests async
- Setup automatique avec `setup.js`
- Coverage sur routes, middlewares et auth
- Exclusion des fichiers de test et node_modules

`setup.js` - Setup global des tests :

- **Mock PostgreSQL** : Simulation complète de la base de données en mémoire
- **Gestion utilisateurs** : Array mockUsers avec compteur d'ID
- **Simulation requêtes** : Implémentation des principales requêtes SQL
- **Reset automatique** : Nettoyage entre chaque test

- **Variables d'environnement** : JWT_SECRET et NODE_ENV=test

Helpers et utilitaires

createTestApp.js - Factory d'application de test :

- Express configuré avec CORS, cookies, JSON
- Routes d'authentification montées
- Configuration CORS pour localhost:3000

authHelper.js - Utilitaires d'authentification :

- **createAuthToken()** : Génération de JWT de test
- **createAuthenticatedUser()** : Création utilisateur avec token
- Gestion des données par défaut et personnalisées

Tests d'authentification

auth.test.js - Tests unitaires complets :

Tests d'inscription :

- Inscription réussie avec données valides
- Rejet champs manquants
- Validation mot de passe faible
- Validation âge minimum (12 ans)
- Gestion email déjà existant

Tests de connexion :

- Connexion réussie avec credentials valides
- Rejet email/mot de passe incorrects
- Validation champs requis
- Vérification cookie authToken

Tests déconnexion :

- Suppression cookie d'authentification
- Message de confirmation

Tests route **/me** :

- Récupération profil utilisateur authentifié
- Validation token JWT
- Retour informations : id, email, isAdmin

Tests changement mot de passe :

- Changement réussi avec validation
- Rejet mot de passe actuel incorrect
- Validation confirmation mot de passe

- Validation complexité nouveau mot de passe
- Interdiction même mot de passe

Tests d'intégration

auth.integration.test.js - Scénarios complets :

Journey utilisateur complète :

1. Inscription utilisateur
2. Connexion et récupération token
3. Accès route protégée **/me**
4. Changement mot de passe
5. Déconnexion
6. Validation ancien mot de passe invalidé
7. Validation nouveau mot de passe fonctionnel

Mock de base de données

Simulation PostgreSQL intelligente :

- **Requêtes INSERT** : Création utilisateurs avec validation unicité email
- **Requêtes SELECT** : Recherche par email et ID
- **Requêtes UPDATE** : Modification mot de passe
- **Gestion erreurs** : Simulation erreurs de contraintes
- **Reset automatique** : État propre pour chaque test

Couverture de tests

Métriques collectées :

- Routes : auth.routes.js et tous les fichiers routes/
- Middlewares : Tous les fichiers d'authentification
- Formats : Text, LCOV, HTML
- Exclusions : node_modules et fichiers de test

Avantages de cette architecture

Isolation : Chaque test s'exécute avec un état propre **Performance** : Mock en mémoire plus rapide qu'une vraie DB **Fiabilité** : Pas de dépendance externe **Maintenabilité** : Helpers réutilisables et configuration centralisée **Couverture** : Tests unitaires + intégration pour validation complète

Frontend

Dépendances

- @angular common, compiler, core, forms, platform-browser, platform-browser-dynamic, router : dépendances permettant le fonctionnement de angular
- @tailwindcss/postcss : tailwind pour utiliser les class dans la mise en forme
- gsap : animations javascript avancée
- heroicons : icons ultra optimisés pour l'appli
- rxjs : outils d'angular pour gérer les requêtes et les données
- zone.js : outils d'angular pour gérer la réactivité de l'app
- tslib : bibliothèque type script

Fonctionnement général de l'application client en Single Page App

Composants

- Site Statique de présentation dans le dossier "components/website"
- Authentification et inscription par les composants "components/login" et "components/register"
- Composants du tableau de bord interaction dans "components/dashboard"
- Composants utilisateurs dans "components/layout" (header et navbar)
- Médias stockés dans le dossier "assets" (vidéos, images, etc.)

Fonctionnement

- Services associés aux composants, assurant la communication avec le serveur Express, conçus selon les principes de modularité et de DRY grâce à l'utilisation de modèles partagés avec les composants. Le service `http` gère les requêtes, le service `errors` traite la gestion des erreurs, et le fichier `environment.ts` centralise les constantes utilisables par l'ensemble des services.
- Application configurée grâce au fichier `app.config.ts` (providers, langue, etc.)
- Routes gérées par le fichier `app.routes.ts` qui utilise des guards pour sécuriser l'accès aux pages
- Responsive : Adaptation mobile/desktop, mobile-first (classes Tailwind adaptatives, sm:, md:, lg:)
- Utilise Router pour les redirections interne
- Import optimisé : Chaque composant importe uniquement ses dépendances
- Lazy loading : Composants chargés à la demande
- Accessibilité : ARIA labels, rôles sémantiques, navigation clavier, A11y (cdk) pour TrapFocus

Fonctionnement des composants "website"

WelcomeComponent

Rôle : Page d'accueil avec animations GSAP et sections d'information

Techniques :

- ViewChild et ViewChildren pour cibler les éléments DOM
- Lifecycle hooks (AfterViewInit, OnDestroy)
- Cleanup automatique des animations ScrollTrigger
- Background vidéo en autoplay avec fallback poster

HeaderComponent

Rôle : En-tête de navigation fixe

Fonctionnalités :

- **Logo cliquable** : Navigation vers l'accueil avec SVG et titre
- **Bouton CTA** : "Accéder aux plans de com" vers le dashboard

Design : Transparent avec overlay sur la vidéo, bouton stylisé avec classe CSS "btn-three"

FooterComponent

Rôle : Pied de page avec liens légaux et réseaux sociaux

Fonctionnalités :

- **Liens légaux** : CGU, Mentions légales, Politique de confidentialité
- **Contact** : Email avec icône de redirection
- **Réseaux sociaux** : Instagram et TikTok avec icônes SVG

FaqComponent

Rôle : Section FAQ avec accordéon interactif

Fonctionnalités :

- **15 questions/réponses** : Couvrant toutes les fonctionnalités de l'app
- **Accordéon animé** : Système toggle avec animations CSS (rotation +, hauteur max)
- **État géré** : Tableau boolean[] pour tracking des éléments ouverts
- **Images démonstratives** : 3 captures d'écran en superposition avec hover effects

Structure : Grid 2 colonnes (images + titre / questions FAQ)

Fonctionnement des services modulaires “http” et “errors”

HttpService

Rôle : Couche d'abstraction centralisée pour toutes les requêtes HTTP

Fonctionnalités :

- **Wrapper HttpClient** : Encapsule le HttpClient Angular natif
- **Configuration centralisée** : baseUrl et defaultOptions depuis environment.ts
- **Méthodes CRUD complètes** : GET, POST, PUT, PATCH, DELETE
- **Construction d'URL** : buildUrl() pour assembler automatiquement baseUrl + endpoint
- **Options par défaut** : withCredentials pour les cookies, headers communs

Architecture :

- **Typage générique** : Méthodes typées `<T>` pour le type safety
- **Réutilisabilité** : Base pour tous les autres services métier

ErrorsService

Rôle : Gestionnaire centralisé des erreurs HTTP et de validation

Fonctionnalités :

- **Gestion différenciée** : Erreurs client vs serveur
- **Mapping des codes HTTP** :
 - 0 : Problème de connectivité
 - 401 : Session expirée → redirection auth
 - 403 : Accès non autorisé
 - 404 : Ressource non trouvée
 - 422 : Validation échouée (email non enregistré)
 - 500 : Erreur serveur interne
- **Messages contextualisés** : Adapte le message selon le type d'erreur
- **Logging centralisé** : Console.error avec détails (status, message, url)
- **Validation métier** : handleValidationError() pour erreurs custom

Architecture :

- **Observable error handling** : Retourne throwError() pour chaînage RxJS
- **Singleton service** : providedIn: 'root' pour instance unique
- **Extensibilité** : Facilite l'ajout de nouveaux types d'erreurs

Fonctionnement des composants liés à l'authentification et l'inscription

AuthService

Rôle : Gestionnaire centralisé de l'authentification et des états utilisateur

Fonctionnalités principales :

- **Signals réactifs** : `isLoggedIn()`, `isAdmin()` pour état temps réel
- **Lifecycle complet** : Login, Register, Logout, CheckAuth
- **Redirection intelligente** : Automatique selon le rôle (admin → /admin, user → /home)
- **Persistence** : Gestion cookies + localStorage pour état application
- **Validation continue** : `checkAuth()` au démarrage pour vérifier validité session

Méthodes clés :

- `login()` : Authentification + redirection + état global
- `register()` : Inscription + redirection login
- `logout()` : Déconnexion + nettoyage complet + redirection
- `changePassword()` : Modification sécurisée mot de passe
- `checkAuth()` : Validation session serveur

AdminService

Rôle : API dédiée aux fonctionnalités d'administration

Fonctionnalités :

- **Statistiques plateforme** : `getAdminStats()` (utilisateurs, campagnes, admins)
- **Gestion campagnes** : `getAllCampaigns()` avec vue globale
- **Héritage HttpService** : Utilise la couche HTTP centralisée
- **Gestion erreurs** : ErrorsService intégré

Composants Authentification

LoginComponent

Rôle : Interface de connexion avec gestion d'état avancée

Fonctionnalités :

- **Formulaire réactif** : ReactiveFormsModule avec validation temps réel
- **État avec Signals** : `errorMessage`, `isLoading` pour UX fluide
- **Redirection automatique** : Effect Angular pour redirection basée sur rôle
- **Validation complète** : Email + mot de passe avec messages d'erreur contextuels
- **Prévention double soumission** : Bouton désactivé pendant traitement

Architecture :

- Injection moderne avec `inject()`
- Effect pour surveillance état auth
- Méthodes utilitaires : `hasError()`, `getError()`, `markAllFieldsAsTouched()`

RegisterComponent

Rôle : Interface d'inscription avec validation avancée

Fonctionnalités :

- **Formulaire complexe** : 6 champs avec validation métier
- **Validators custom** :
 - `minimumAgeValidator()` : Âge minimum 13 ans
 - `strongPasswordValidator()` : Complexité mot de passe
 - `passwordMatchValidator()` : Correspondance confirmation
- **Validation temps réel** : Messages d'erreur instantanés
- **UX optimisée** : Gestion états loading, erreurs, accessibilité

Validations métier :

- Nom/Prénom : 2 caractères minimum
- Date naissance : Âge minimum + date valide + pas future
- Email : Format email valide
- Mot de passe : 8 caractères + majuscule + minuscule + chiffre + caractère spécial
- Confirmation : Correspondance exacte

AdminComponent

Rôle : Dashboard administrateur avec vue globale plateforme

Fonctionnalités :

- **Vue statistiques** : Cards avec métriques clés (campagnes, utilisateurs, admins)
- **Liste campagnes** : Table complète avec ID, nom, dates création/modification
- **Gestion état** : Loading, erreurs, rafraîchissement
- **Actions admin** : Rafraîchir données, déconnexion

Système de Guards

adminGuard

Rôle : Protection routes administrateur

Logique :

- Vérifie authentification + statut admin via `checkAuth()`
- Utilisateur admin → Accès autorisé
- Utilisateur standard → Redirection `/home`
- Non authentifié → Redirection `/login`

userGuard

Rôle : Protection routes utilisateur standard

Logique :

- Vérifie authentification + exclut admins
- Utilisateur standard → Accès autorisé
- Administrateur → Redirection [/admin](#)
- Non authentifié → Redirection [/login](#)

Configuration Routes

Protection Cascade

- **Guards** vérifient auth + rôle
- **Redirections automatiques** selon statut
- **Enfants protégés** : /home avec sous-routes (recap, goals, targets, etc.)

Gestion d'Erreurs

Types d'Erreurs Gérées

- **401** : Session expirée → Redirection login
- **403** : Accès non autorisé
- **422** : Email non enregistré (message spécifique)
- **500** : Erreur serveur
- **Validation** : Messages contextuels par champ

UX des Erreurs

- Messages utilisateur friendly
- États loading pour feedback immédiat
- Validation temps réel pour prévention erreurs

Fonctionnement des composants de gestion d'utilisateur "layout"

HeaderComponent (Orchestrateur Principal)

Rôle : Hub central gérant utilisateur, campagnes, modals et état global

Responsabilités principales :

- **État global** : Gestion utilisateur (nom, prénom), campagnes, invitations
- **Orchestration modals** : Via ModalService centralisé (campaign, profile, settings, delete)

- **Communication parent-enfant** : Outputs vers app principale (logout, campaignSelected, accountDeleted)
- **Persistence** : LocalStorage pour currentCampaign et currentCampaignId
- **Données temps réel** : forkJoin pour chargement initial (user + campaigns + invitations)

Composants enfants gérés :

- CampaignSelectorComponent (sélection campagne)
- UserMenuComponent (menu utilisateur)
- 4 modals spécialisées

HeaderService (API Layer)

Rôle : Interface unique pour toutes les opérations utilisateur/campagne

Endpoints couverts :

- **Profil** : `getUserName()`, `updateProfile()`
- **Campagnes** : `getCampaigns()`, `createCampaign()`, `deleteCampaign()`
- **Collaboration** : `inviteUser()`, `getCampaignUsers()`, `revokeUserAccess()`
- **Invitations** : `getPendingInvitations()`, `acceptInvitation()`, `rejectInvitation()`
- **Compte** : `deleteAccount()`, `leaveSharedCampaign()`

Architecture : Hérite HttpService + ErrorsService pour standardisation

CampaignSelectorComponent

Rôle : Interface de sélection/affichage campagne courante

Fonctionnalités :

- **Affichage dynamique** : Titre adapté selon état (campagne sélectionnée vs "Créer ou choisir")
- **Dropdown hover** : Menu avec campagnes existantes + bouton "Nouvelle campagne"
- **États visuels** : Loading spinner, messages d'erreur/succès

UserMenuComponent

Rôle : Interface utilisateur avec indicateurs visuels

Fonctionnalités :

- **Photo profil** : Image statique avec fallback

- **Nom cliquable** : Ouvre modal profil au clic
- **Bouton paramètres** : Badge notification pour invitations pendantes (avec animation pulse)
- **Bouton déconnexion** : Avec icône Material

CampaignModalComponent

Rôle : Création de nouvelle campagne

Caractéristiques :

- **Validation temps réel** : 3-100 caractères, compteur caractères
- **Prévention double soumission** : Signal `isSubmitting`
- **Focus management** : Auto-focus + restauration focus précédent
- **Sanitisation** : Nettoyage caractères dangereux

ProfileModalComponent

Rôle : Modification profil + changement mot de passe

Architecture dual :

- **Section profil** : Modification prénom/nom avec validation longueur
- **Section mot de passe** : Collapse/expand avec validation complexité
- **États séparés** : `loadingState` et `passwordLoadingState` indépendants
- **Validation avancée** : Regex mot de passe, correspondance confirmation
- **UX optimisée** : Auto-clear messages succès, focus management

SettingsModalComponent

Rôle : Gestion campagnes, invitations, et administration compte

Sections principales :

- **Invitations pendantes** : Accept/reject avec reload automatique
- **Gestion campagnes** : Vue propriétaire vs partagée, actions contextuelles
- **Formulaire invitation** : Email + rôle (reader/editor) avec validation
- **Liste utilisateurs** : Par campagne avec révocation accès
- **Zone danger** : Suppression compte

Logique métier :

- Protection campagne active (impossible supprimer/quitter si sélectionnée)
- Chargement lazy des utilisateurs par campagne
- Computed signals pour statistiques (total, owned, shared)

Rôle : Modal réutilisable pour confirmations destructives

Personnalisation :

- **Inputs configurables** : title, message, buttonText, buttonClass, icons
- **États loading** : Spinner + texte adaptatif
- **Options accessibilité** : preventBackdropClose, preventEscClose, autoFocus
- **Gestion focus** : Piégeage + restauration automatique

ModalService

Rôle : Gestionnaire centralisé d'état des modals

Fonctionnalités :

- **État global** : Signal states() pour toutes les modals
- **Méthodes toggle** : open/close pour chaque modal
- **Raccourcis clavier** : Handler global pour Échap, etc.
- **Prévention conflits** : Assure qu'une seule modal ouverte

Fonctionnement des composants liés aux fonctions du tableaux de bord (gestion des tâches, du calendrier, etc.)

HomeComponent (Layout Principal)

Rôle : Container principal du dashboard avec layout fixe

Structure :

- **NavbarComponent** : Navigation latérale (collapse mobile)
- **HeaderComponent** : Barre supérieure avec gestion utilisateur/campagnes
- **RouterOutlet** : Zone de contenu dynamique pour les composants métier
- **Layout responsive** : Adaptation mobile avec **lg:ml-62** pour décalage navbar

Responsabilité : Orchestration logout centralisé via AuthService

Services Métier

ActionsService (Gestion Calendrier)

Rôle : Interface API pour événements/actions de campagne

Endpoints :

- **getEventsWithAccess()** : Événements + permissions campagne (principal)
- **getEvents()** : Événements seuls (compatibilité)
- **addEvent(), updateEvent(), deleteEvent()** : CRUD complet
- **Validation dates** : Conversion ISO string automatique
- **Gestion permissions** : Contrôle granulaire par campagne

GoalsService (Gestion Objectifs)

Rôle : Interface API pour objectifs avec upload d'images

Fonctionnalités :

- `getCampaignPermissions()` : Vérification droits avant actions
- `uploadGoalImage()` : Upload avec FormData + validation type/taille
- `getGoalsByCampaignId()`, `createGoal()`, `updateGoal()`, `deleteGoal()` : CRUD
- **Images sécurisées** : Authentification par cookies + middleware backend
- **Validation** : ID campagne requis pour toutes opérations

TargetsService (Gestion Cibles)

Rôle : Identique à GoalsService mais pour les cibles

Structure : Même pattern que Goals (permissions + upload + CRUD)

TaskService (Gestion Tâches)

Rôle : Service avec état réactif centralisé (Signals)

Architecture avancée :

- **Signals état** : `tasks`, `permissions` avec computed dérivés
- **Computed automatiques** : `todoTasks`, `inProgressTasks`, `doneTasks`, `isReadOnly`
- **État persistant** : Survit aux changements de route
- **Méthodes réactives** : `fetchTasks()`, `addTask()`, `updateTaskStatus()`, `updateTask()`, `deleteTask()`
- **Gestion permissions** : Alerts automatiques si lecture seule

Composants Métier

ActionsComponent (Calendrier Interactif)

Rôle : Calendrier avec événements et modals de gestion

Fonctionnalités principales :

- **Calendrier custom** : Grid CSS responsive avec navigation mois
- **Liste événements** : Sidebar avec tri chronologique
- **Modals CRUD** : Ajout/édition/suppression avec validation
- **Permissions dynamiques** : UI adaptée selon droits (boutons conditionnels)
- **Cache local** : Fallback localStorage si erreur réseau
- **Focus management** : Auto-focus + ViewChild pour modals

État réactif :

- Signals pour événements, dates, modals
- Computed pour permissions et validation
- Effects pour cleanup et gestion modals

GoalsComponent (Gestion Objectifs)

Rôle : Interface CRUD pour objectifs avec upload d'images

Fonctionnalités :

- **Cards système** : Composant GoalsCardsComponent réutilisable
- **Upload drag&drop** : Validation type/taille + preview instantané
- **Formulaires réactifs** : Validation temps réel avec FormBuilder
- **Permissions UI** : Boutons/modals conditionnels selon droits
- **États multiples** : Loading séparé pour ajout/édition
- **Focus automatique** : ViewChild + setTimeout pour UX fluide

Patterns techniques :

- forkJoin pour chargement parallèle (goals + permissions)
- FormData pour uploads avec méta-données
- TrackBy fonctions pour performances listes
- Mapping Goal → GoalCard pour séparation préoccupations

TargetsComponent (Gestion Cibles)

Rôle : Clone de GoalsComponent adapté aux cibles

Spécificités : Terminologie "cibles/sous-cibles" vs "objectifs/sous-objectifs"

TasksComponent (Tableau Kanban)

Rôle : Interface Kanban avec drag&drop

Fonctionnalités avancées :

- **CDK Drag&Drop** : 3 colonnes connectées (À faire, En cours, Terminé)
- **Modals inline** : Ajout/édition avec status pré-sélectionné par colonne
- **Permissions granulaires** : Drag désactivé + boutons masqués si lecture seule
- **Intégration service** : Consomme directement signals TaskService
- **Effects réactifs** : Rechargement auto si campagne change

UX patterns :

- Boutons + par colonne pour création directe
- Boutons édition au hover sur chaque carte
- Alerts utilisateur pour tentatives non autorisées
- Focus management sur modals

RecapComponent (Dashboard Résumé)

Rôle : Vue d'ensemble avec derniers éléments de chaque section

Logique métier :

- **Chargement parallèle** : forkJoin pour goals, targets, events
- **TaskService integration** : Effect pour écouter tâches en cours
- **Dernier élément** : Tri par ID/date + prise du dernier
- **Navigation contextuelle** : Boutons redirection vers sections complètes
- **États vides** : Messages encouragement + boutons action

Optimisations :

- Loading states séparés par section
- Mapping adapté pour affichage cards
- Gestion erreurs granulaire

Composants Réutilisables

GoalsCardsComponent & TargetsCardsComponent

Rôle : Cards d'affichage avec bouton édition conditionnel

Fonctionnalités :

- **Input signals** : title, imageUrl, description, items, canEdit
- **Output events** : edit pour communication parent
- **Performance** : ChangeDetectionStrategy.OnPush + TrackBy
- **Fallback images** : Base64 placeholder si erreur chargement

Patterns Architecturaux Transversaux

Gestion des Permissions

Principe : Vérification systématique avant toute action

Implémentation :

- `getCampaignPermissions()` en parallèle du chargement données
- Computed signals pour `canEdit()`, `canAdd()`
- UI conditionnelle avec `@if` Angular 17
- Messages explicites pour restrictions

Gestion d'État

Signals partout : Remplacement observables pour état local

Avantages :

- Réactivité automatique
- Computed dérivés performants
- Effects pour side-effects
- Meilleure lisibilité code

Upload de Fichiers

Pattern standardisé :

- Drag&drop + input file
- Validation type/taille côté client
- FormData avec métadonnées
- Preview instantané via FileReader
- Gestion erreurs explicites

Focus Management

UX soignée :

- ViewChild pour référence éléments
- setTimeout(100ms) pour ouverture modals
- Auto-focus premier champ

Gestion des Erreurs

Approche utilisateur :

- Messages contextuels par action
- Clear error explicite
- Fallback localStorage pour resilience
- Loading states pour feedback

Performance

Optimisations :

- OnPush strategy sur composants purs
- TrackBy functions pour For
- forkJoin pour chargements parallèles
- Computed signals pour calculs dérivés
- takeUntil pattern pour cleanup subscriptions

Fonctionnement des tests unitaires du service "auth"

- **Setup** : Providers Angular 17+ (`provideHttpClient`, `provideHttpClientTesting`)
- **Mocks** : Router et TaskService avec `jasmine.createSpyObj`
- **HttpTestingController** : Validation requêtes + simulation réponses
- **État initial** : Mock `checkAuth()` du constructeur

Tests Principaux

login()

- **Succès utilisateur** : POST + signals updated + redirection `/home` + localStorage cleared
- **Succès admin** : Même logique + redirection `/admin` + `isAdmin: true`
- **Erreur** : Status 401 → signals false + message "Échec connexion"

register()

- **Succès** : POST + redirection `/login`
- **Erreur backend** : Message spécifique propagé
- **Erreur générique** : Message par défaut

logout()

- **Succès** : POST + signals false + `clearTasks()` + localStorage cleaned + redirection `/login`
- **Erreur serveur** : Même nettoyage en fallback

checkAuth()

- **Authentifié** : GET `/me` + signals updated selon `isAdmin`
- **Non authentifié** : Status 401 + signals false + error "Non authentifié"

changePassword()

- **Succès** : POST avec `ChangePasswordData`
- **Erreur** : Message backend ou par défaut propagé

Méthodes utilitaires

- `isUserLoggedIn()`, `isUserAdmin()`, `isAuthChecked()` : Lecture signals

clearLocalStorage()

- **Nettoyage sélectif** : Supprime clés spécifiques, préserve autres
- **Test indirect** : Via `login()` qui appelle méthode privée

Patterns de Test

- **AAA Pattern** : Arrange, Act, Assert
- **Mocking** : `routerSpy.navigate`, `taskServiceSpy.clearTasks`
- **Signals testing** : Accès privé via `service['signalName']`
- **HTTP simulation** : `expectOne()` + `flush()` + `verify()`
- **localStorage** : `clear()` avant/après tests